本体落地对金融可信 AI 的挑战

语义映射与对象存储的取舍

演讲嘉宾：徐德华

天翼征信有限公司 总经理助理

目录

CONTENT

01 老分叉：OLTP / OLAP 的岔路

02 先分层：口径一份，存取两种

03 两条路：各有优劣

04 分情况：什么需求走哪条路

05 共同点：供数、验收与踩坑

01 老分叉

智能体又走回了 OLTP / OLAP 的岔路

数据访问的老分叉：同一份数据，两种负载

分叉的根源是负载不同，不是技术偏好

	OLTP	OLAP
面向	单个业务对象	大量记录的聚合
典型问题	这份合同现在什么状态?	上季度各地区交易额合计?
查询形态	点查 + 少量关联	大范围扫描 + 分组聚合
时延	毫秒	秒到分钟
写入	高频小写入，状态随时变	批量追加，基本只读
数据模型	规范化实体与关系	宽表 / 星型 + 指标口径

历史上发生了什么

没有一个系统能同时把两种负载做好

↓

分库 + ETL

↓

报表和业务系统「对不上数」

↓

统一指标层 / HTAP

智能体把两种负载塞进了同一个任务

示例：一个关联研判任务，抽象成对象与关系，每一步标出访问模式

1 偏离检测

哪些交易价格明显偏离同类均值?

OLAP

全量扫描 + group by
结果是聚合值

2 关联穿透

交易双方有没有股权或人员关联?

多跳遍历

沿关系走 3–5 跳
中间结果随度数指数增长

3 状态读取

合同签了没有、距发起隔了几天?

OLTP 点查

按主键读当前状态

4 结果回写

标记待核查，派发复核

写入

新增一条带来源的事实
（状态 + 证据）

一次研判 = 分析 + 穿透 + 状态 + 回写　人在 BI 和业务系统里分开做，智能体要在一个上下文里连着做

所以做本体 + 智能体之前，先拆目标任务，看清负载画像

猜想：本体的分叉，也许是老分叉在语义层的重演

本体是一层「翻译」，还是一份「数据」？

语义映射

语义在查询时绑定

本体是一层元数据
问题翻译成对数仓的 SQL，数据不搬家

OLAP 血统 · 近似指标层 / 语义层

对象存储

语义在写入时绑定

实例、关系、状态物化成对象
智能体直接操作对象

OLTP 血统 · 代表：Palantir Ontology

类比的边界： OLTP / OLAP 分的是负载，本体分叉分的是语义何时绑定到数据——负载决定绑定时机
留一个问题： 什么样的需求，该走哪条路？

02 先分层

口径只能有一份，存取可以有两种

两层各管一件事：概念层管对不对，实例层管快不快

OLTP / OLAP 分库后最大的麻烦不是性能，是口径对不上——所以口径只留一份

概念层 · 唯一一份

管：对象 · 属性 · 关系 · 口径 · 规则 · 决策逻辑
变化：低频，按版本发布　**谁改：**业务专家 + 建模师
验收：口径一致、可追溯

同一份口径生成两种实现

实例层 · 可以有两种实现

语义映射
查询时翻译成 SQL

对象存储
写入时物化为对象

管：亿级实例 · 实时状态 · 历史链路 · 结果回写
变化：高频，系统自动同步　**验收：**查得到、查得快、查得准

例子：口径要带生效时点

一个金额门槛类口径，新规则生效前后不同。判断一笔历史业务，要按「业务发生时」匹配口径版本，而不是用最新版。

实现上：
口径定义带 valid_from / valid_to
查询时按事实发生时间做 as-of 匹配

混做的两种结局：一个例子

结局 A：实例塞进知识库

图谱膨胀 → 推理变慢 → schema 不敢改

结局 B：概念跟着表结构走

换一张底表 → 口径跟着变 → 智能体无感知

一个常见的演化：从一层到两层

起步：一层

「企业—股东—交易」的定义和实例放在同一个图谱，规则直接写在实例上

规模小时很顺手——早期这样最快

→

实例到千万级

改一次「关联方」定义 → 全量重跑推理

同一口径被多个场景各复制一份

→

拆成两层

定义只改一处

实例层只管算和存

口径版本可追溯

03 两条路

语义映射与对象存储各有优劣

正面对比：各自赢在哪、输在哪

映射输在穿透和回写，对象存储输在一致性和成本——高亮的是相对占优的一侧

维度	语义映射	对象存储
上线速度	**快，周级**	慢，要建同步链路
存储冗余	**无**	有，全量或核心子集
多跳穿透	弱，跳数一多 SQL 爆炸	**强，关系已物化**
查询时延	取决于数仓，秒到分钟	**毫秒到秒**
状态与回写	无处安放	**原生支持**
口径独立性	受底层表结构牵制	**独立，但要对账**
一致性	**天然一致**	同步延迟、增量出错
真值归属	**源系统**	要明确写死
执行位置	下推到数仓引擎	对象库 / 图引擎本地执行
Schema 演进	**改映射规则即可**	迁移 + 回灌历史数据
权限与审计	**继承源库权限**	要在对象层重建行列级权限

代价付在哪一端：读的时候，还是写的时候

同一类权衡：范式 vs 反范式，视图 vs 物化视图

语义映射

源数据 → 语义 → **智能体**

每次读都付

翻译 + 跨表关联 + 现场计算

k 跳 ≈ k 次自连接，中间结果 ≈ 度数^k

对象存储

源数据 → 语义 → 智能体

每次写都付

同步链路 + 存储 + 增量一致性

写放大 ≈ 依赖扇出；读只付访问到的边

对象存储绕不开：物化结果可能短暂过期

✗ 不带状态

股东已变更，对象库尚未同步完

「两者无关联」

一个错误的确定性结论

✓ 带口径状态

变更后标记「收敛中」，收敛前不给确定性结论

「暂未发现关联，股权数据同步中」

监管这类要求可追溯的场景，差别就在可信与否

04 分情况

什么样的需求适合哪条路

我们用来区分情况的六个维度

维度	偏向语义映射	偏向对象存储	背后的原因
时效要求	T+1、小时级可接受	秒级、实时	映射的时延受数仓现场计算拖累
数据量	明细量大，只要聚合结果	核心对象可控，关系密集	物化的存储和同步成本随数据量增长
查询形态	扫描 + 聚合	点查 + 多跳（≥ 3 跳）	多跳在 SQL 上是多次 join
回写需求	只读	要记录状态、处置结果	映射没有地方放新产生的事实
变更频率	源表结构、口径常变	结构稳定	物化后每次变更都要重算回灌
读写比	查询少、问题分散	同一批对象被反复查询	物化是用一次写入换多次读取

一个粗略的直觉

重复读取次数 × 每次现场计算的代价 ≫ 变更次数 × 每次变更要刷新的范围 → 物化划算；反之映射更省

一张情况地图



实线：比较有把握
左下、右上两个象限

虚线：待讨论
另外两个象限，后面细说

怎么用

把自己的智能体任务拆开，每一步点到图上，看点主要落在哪

两种比较有把握的情况

用「可能更合适」而不是「应该」——这是我们目前的经验，不是定律

情况一：分析为主、口径常变

可能更合适	**语义映射**
典型需求	「上季度各分支机构某指标同比」 扫描 + group by
为什么	问题分散，结果是聚合值；数据不搬家，口径一改立即生效
前提	映射规则本身版本化
要留意	第一个稳定出现的多跳需求，可能是负载开始迁移的信号

情况二：穿透与处置为主

可能更合适	**对象存储**
典型需求	「A 的实际控制人还控制了哪些与 A 同场出现的主体」 k 跳遍历 + 集合求交
为什么	3–5 跳反复查询、要秒级返回；研判结果要回写成新的事实
前提	核心对象边界清楚、数据量可控、真值归属写清楚
要留意	同步链路的运维成本，以及口径状态标注

灰色地带：我们还没想清楚的几种情况

?

多跳但离线，数据量很大（百亿级关系）

目前的倾向

映射 + 预计算部分路径

不确定的地方

物化成本很高、查询频率又低——够不够？

?

要多跳，口径又常变

目前的倾向

只物化关系骨架，口径计算留在查询时

不确定的地方

每次口径调整都要回灌吗？骨架和口径怎么切？

?

两种负载都很重

目前的倾向

先确认是不是真的两种都重

不确定的地方

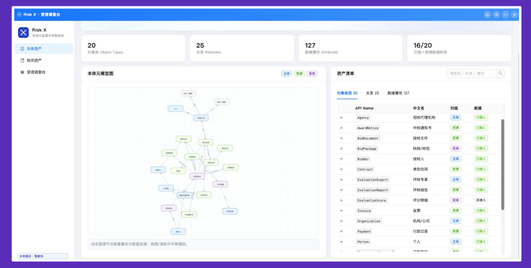
两套都建、一个入口——两套链路和持续对账的代价值得吗？

这一页想和大家交流：灰色地带里，你们是怎么选的？

案例一：一个监管场景的本体实践

四个模块，按访问模式看

本体语义基座



过滤 / 点查 + 多跳遍历

统一对象定义；按属性取数、沿关系探查

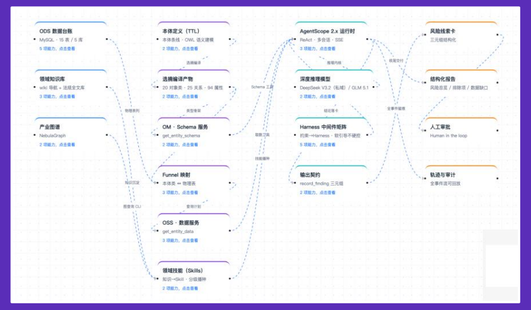
LLM Wiki



as-of 检索

文本知识编译成可链接条目，按生效时点和适用范围匹配

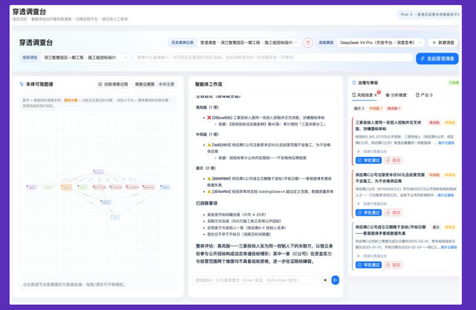
推理引擎（OAG）



受控调用

智能体只发受控调用，长链检索和确定性计算交给引擎

统一分析入口



轨迹 + 回写

记录推理轨迹与证据来源，人工反馈作为新事实回写

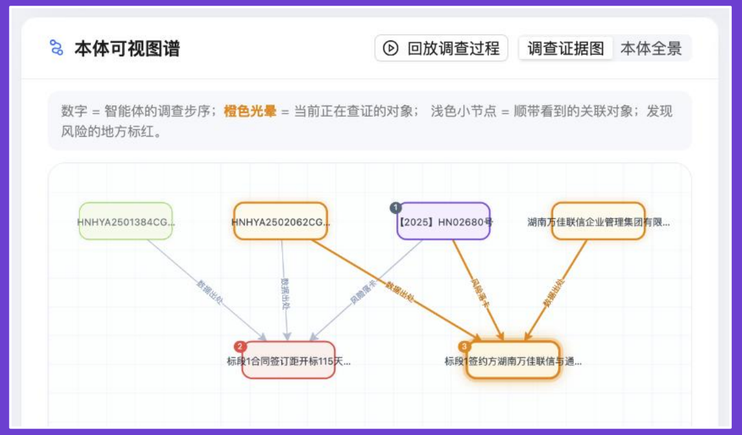
对应 P5 任务链

偏离检测 扫描聚合 → **关联关系** 多跳遍历 → **签约时点** 点查状态 → **风险落卡** 回写

案例二：一次研判怎么走完

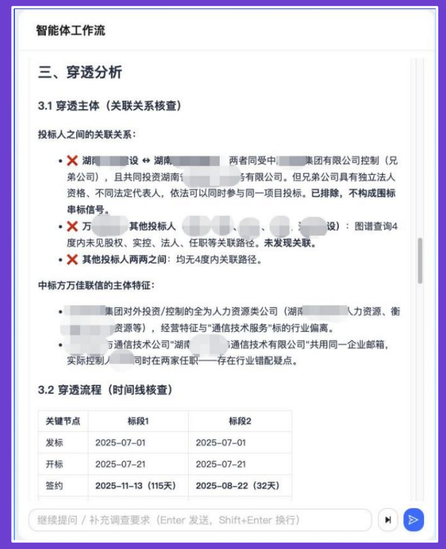
① 沿 schema 探索路径

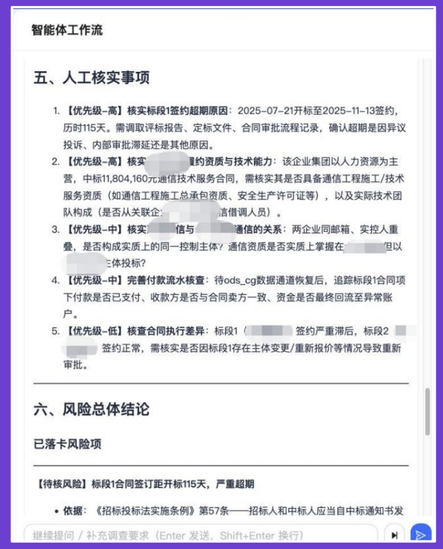
读元数据决定走哪些关系



② 按关系类型分别多跳遍历

主体、流程、资金...结果在对象层合并





③ 结论写回为新对象

结论 + 证据链 + 条文依据



4 类 5 子类 → 12 类 42 子类

风险发现类型：补足既有规则未覆盖的风险类型

+80%

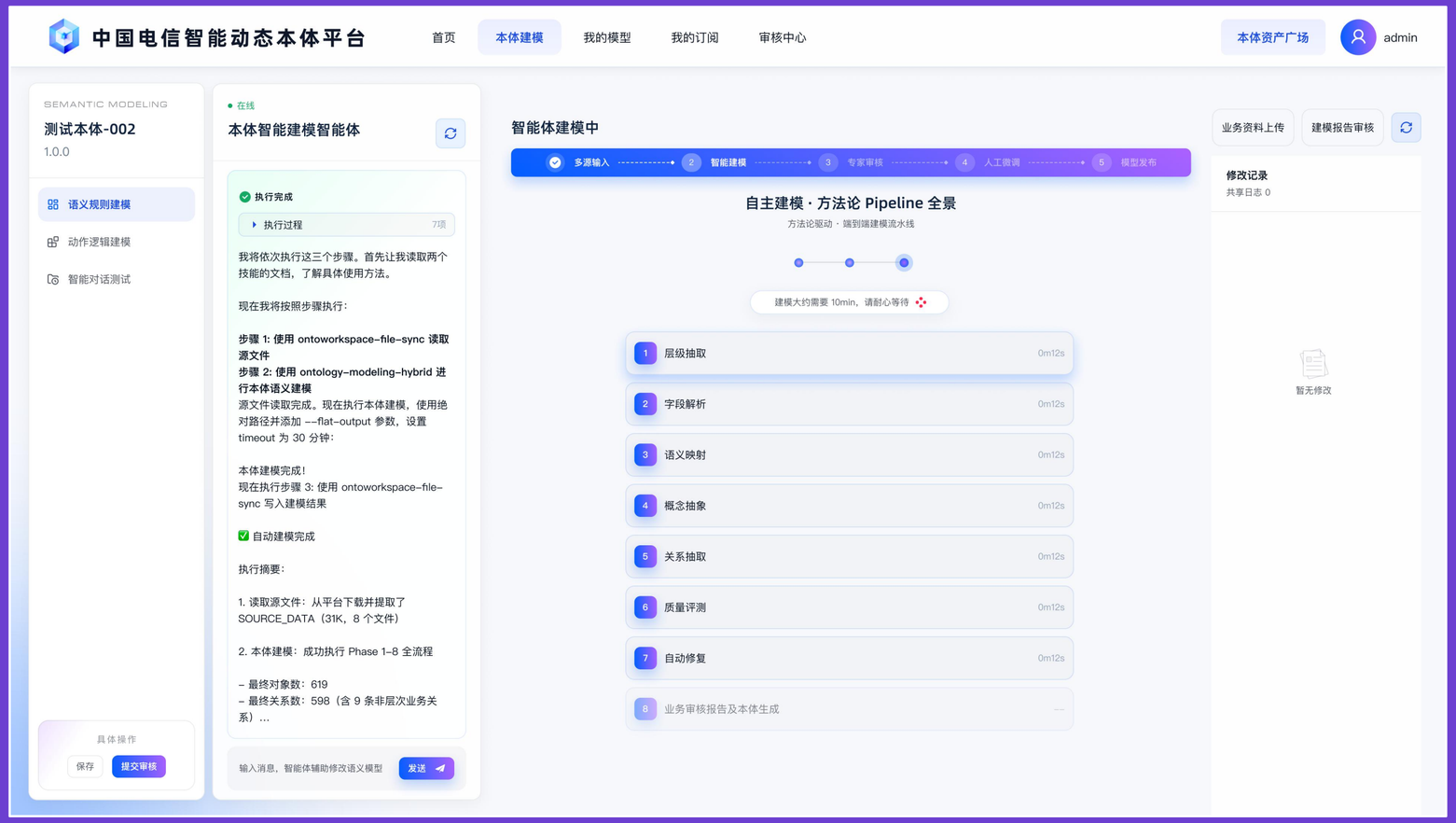
关键事实判断准确度
（相较纯大模型方案）

可信的工程含义

每个结论都能沿 provenance 回到源数据

案例三：从单场景到平台

中国电信 星海 · 智能动态本体平台



组件式服务

- onto-schema
- onto-wiki
- onto-graph
- onto-FL
- onto-loop
- onto-import
- onto-domain
- onto-monitor

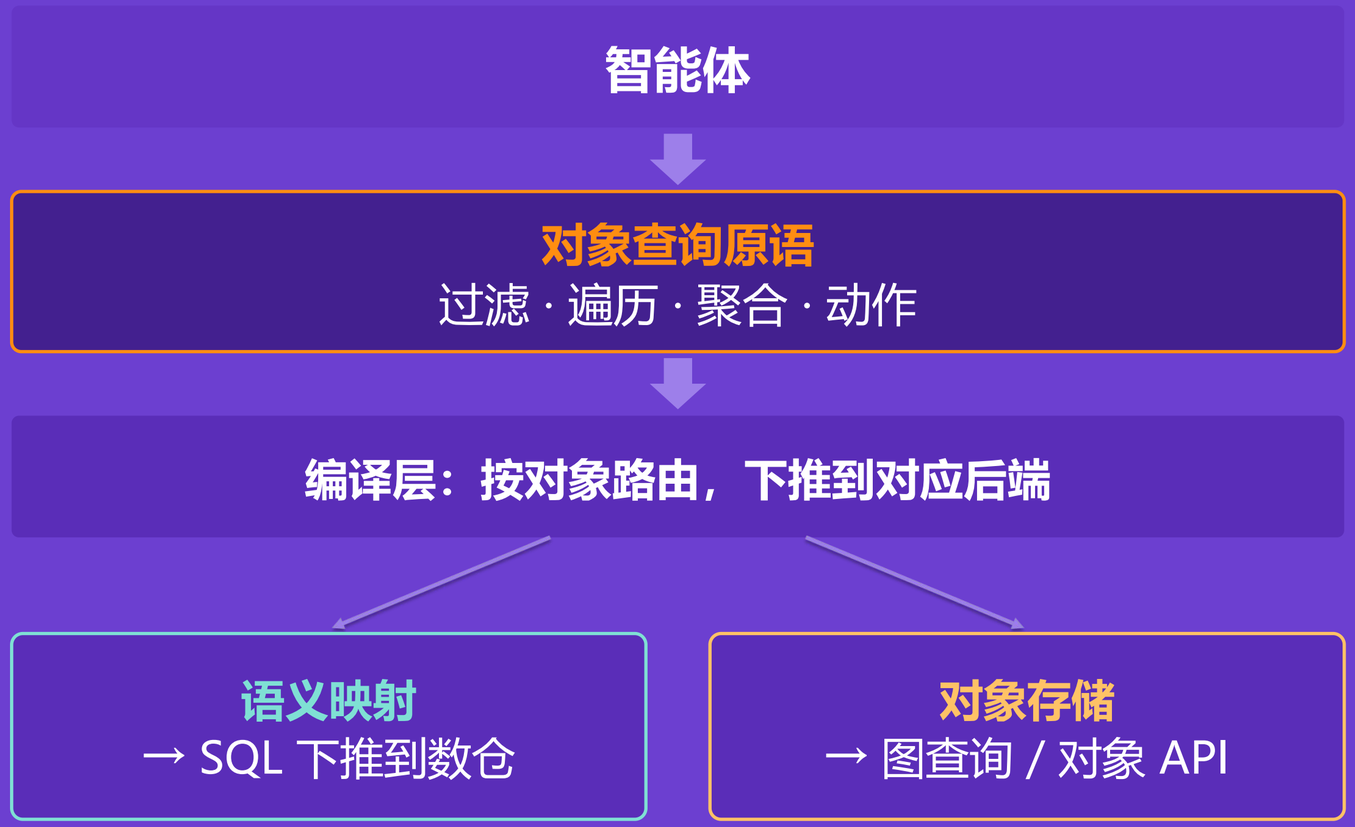
单域跑通后，把能力拆成组件：下一个域从组件起步，而不是从零起步

05 共同点

供数、验收与踩坑

对智能体来说，接口最好长得一样

底层走哪条路不让智能体感知——将来改道，智能体侧不用重写



开放问题：接口统一能把「不可逆」降低到什么程度？我们还在观察

一次供数的四步

1. **返回概貌**
   本体先返回对象概貌（元数据），智能体选主对象
2. **按任务剪枝**
   只给最小必要的属性和关系
3. **受治理取数**
   只调原语，不自己拼 SQL、不自己遍历图
4. **明确降级**
   上下文不足、超时或口径收敛中 → 人工确认或明确告知

怎么证明建得比原来好：交付物与指标

每个业务域交付五件套

本体片段 含实现方式：物化 / 映射

规则库 带版本和生效时点

案例库 已核实的正反案例

已验证查询 高频问题的标准查询

评测集 专家金标准 + 盲评，保留 hold-out

每个正向指标，配一个反向指标

正向指标	配对的反向指标
研判准确率	单次任务成本 / 时延
风险发现数	误报率 / 人工复核工时
穿透查询时延	同步延迟 / 存储成本
对象覆盖数	口径冲突数

跨域复用率怎么算

✗ 复用对象数 / 总对象数

容易虚高

✓ **第 N 个域的交付人力相对第 1 个域下降多少**

常见的坑

重点讲「当时为什么会这么做」——这比解法更有参考价值

坑	当时为什么会这么做	现在的做法
概念与数据混建	早期规模小，一层最快	先拆两层，口径只留一份
只看到问数需求就定了实现	立项时只有分析类需求	先拆目标任务，看清负载再选
物化结果静默过期	默认同步链路可靠	失效传播 + 口径状态标注
把「混合」当默认答案	觉得两边都要更稳妥	先确认两种负载是否真的都重
建成了证明不了	验收时只有演示	金标准题库 + hold-out + 反向指标

小结，以及留给大家的几个问题

我们目前的理解

1. 口径只能有一份，存取可以有两种
2. 语义映射与对象存储各有优劣：成本一个付在读，一个付在写
3. 选哪条，取决于时效、数据量、查询形态、回写、变更频率和读写比
4. 「可信」落到工程上是三件事：口径唯一、结论可追溯、过期不静默

留给现场的问题

智能体的负载，会不会随着用法成熟而迁移——从问数走向研判？

统一接口，能把「不可逆」降低到什么程度？

灰色地带里，大家是怎么选的？

THANKS

演讲嘉宾：徐德华

天翼征信有限公司 总经理助理